```
class Car extends Vehicle
{
    public function getMaxPassengers(): string
    {
        // NumberFormatter requires the intl extension
        $formatter = new NumberFormatter('en', NumberFormatter::SPELLOUT);
        return $formatter->format($this->maxPassengers);
    }
}
```

```
Output:
Fatal error: Declaration of Car::getMaxPassengers(): string must be
compatible with Vehicle::getMaxPassengers(): int
```

Score another point for declaring return types: if none had been specified in the parent declaration, we would be free to change the return type in the child and introduce potential bugs in client code. Finally, we cannot add or remove arguments to overloaded methods.

```
    // new declaration for Vehicle::getMaxPassengers()
    public function getMaxPassengers() {}

    // Car::getMaxPassengers() with an argument...
    public function getMaxPassengers($behaviour)
```

```
Output:
Fatal error: Declaration of Car::getMaxPassengers($behaviour) must be
compatible with Vehicle::getMaxPassengers()
```

For older or less strict code, there is the option of declaring class members as `final`. Only methods, constants, or the class itself may be declared as final, and pre-PHP 8.1 private methods could not be declared in this way. `final` method and constants cannot be overloaded, and `final` classes cannot be extended (though of course that isn't relevant to the LSP).

```
    // new declaration for Vehicle::getMaxPassengers()
    final public function getMaxPassengers() {}

    // Car::getMaxPassengers()
```